Ενότητα 02:
Εντολές:
Η Γλώσσα της Μηχανής

ΕΕΙ 233: ΟΡΓΑΝΩΣΗ ΥΠΟΛΟΓΙΣΤΩΝ

ΑΝΔΡΕΑΣ ΔΙΑΒΑΣΤΟΣ

andreas.diavastos@cut.ac.cy

Τεχνολογικό Πανεπιστήμιο Κύπρου | Τμήμα Ηλεκτρολόγων Μηχανικών και Μηχανικών Ηλεκτρονικών Υπολογιστών και Πληροφορικής

CARL

1

Τεχνολογικό Πανεπιστήμιο Κύπρου | Τμήμα Ηλεκτρολόγων Μηχανικών και Μηχανικών Ηλεκτρονικών Υπολογιστών και Πληροφορικής

Outline

- Instruction Set Architecture
- MIPS Architecture
- Έννοια αποθηκευμένου προγράμματος
- ARM and x86 ISAs

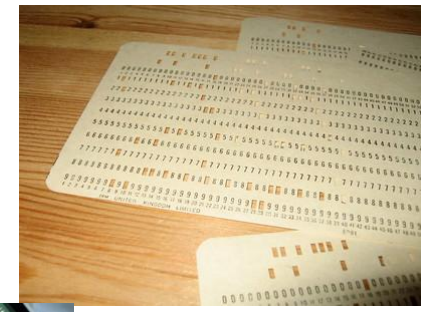
- Υλικό από Κεφάλαιο 2 του βιβλίου του μαθήματος

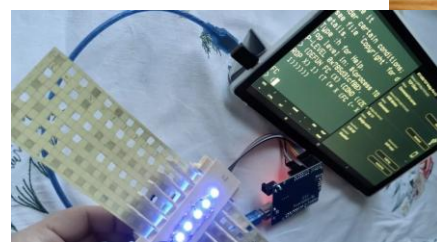
2

Programming Punch Cards

Punch Card in Punch Card Machine

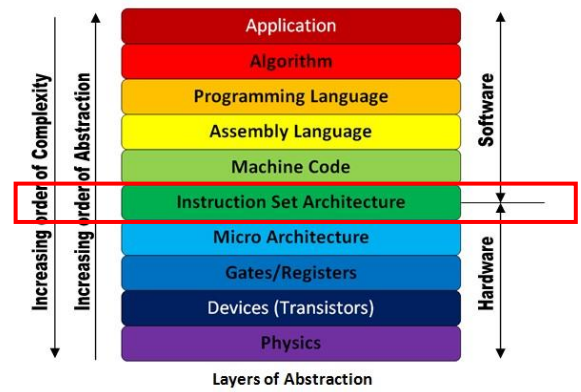






Instruction Set Architecture (ISA)

- ISA είναι η διεπαφή (interface) μεταξύ υλικού και λογισμικού. Καθορίζει πώς το λογισμικό ελέγχει το υλικό
- **Σκοπός:** Παρέχει αφαιρετικότητα στους προγραμματιστές για να αλληλοεπιδρούν με ένα υπολογιστικό σύστημα
- Η ISA ορίζει:
  - Εντολές που μπορεί να εκτελέσει ο επεξεργαστής
  - Μηχανισμούς μεταφοράς δεδομένων & τρόπο πρόσβασης στα δεδομένα
  - Μηχανισμούς ελέγχου (branch, jumps, κλπ)
  - Συμβόλαιο μεταξύ προγραμματιστή/compiler και υπολογιστικού συστήματος (Hardware)
- **Πλεονέκτημα:** Πολλές διαφορετικές υλοποιήσεις της ίδιας ISA μιλούν την ίδια γλώσσα
- **Μειονέκτημα:** Το ISA δεν είναι συμβατό μεταξύ μικροεπεξεργαστών από διαφορετικές οικογένειες

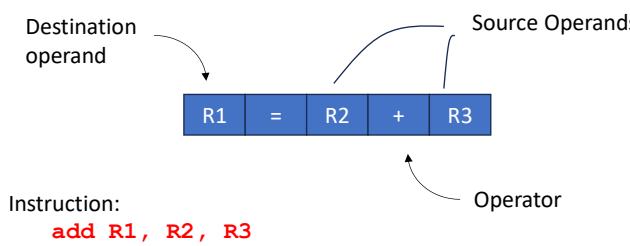


Instruction Set Architecture (ISA)

- Επίπεδα ISA:
  - **Υλικού**: Καθορίζει τον τρόπο με τον οποίο ο επεξεργαστής εκτελεί τις εντολές
  - **Λογισμικού**: Καθορίζει τις εντολές που είναι διαθέσιμες για προγραμματισμό
- Σημασία:
  - Καθορίζει τη δομή των εντολών
  - Ο σχεδιασμός ISA επηρεάζει την επίδοση (performance), την απόδοση ισχύος (power efficiency) και τη συμβατότητα (compatibility)

Παράδειγμα δομής μιας εντολής

- Μια εντολή έχει:
  - Ένα operator (op ή opcode)
  - Ένα ή δύο operands εισόδου (Source operands - src)
  - Ένα operand εξόδου/αποτέλεσμα (Destination operand - dest)



Instruction:

```
add R1, R2, R3
```

Σύνταξη εντολών

- Intel x86
  - `op  dest, src1, src2`
  - `add R1, R2, R3`
- ARM
  - `op  register, src or dest`
  - `add R1, R2, R3`
- MIPS
  - Mostly → `op  dest, src1, src2`
  - `add R1, R2, R3`

Key ISA Decisions

- Λειτουργίες:
  - Πόσες?
  - Ποιες?
- Operands:
  - Πόσοι?
  - Τύπος δεδομένων?
- Addressing mode
  - Που βρίσκονται τα δεδομένα?
- Εντολές:
  - Μέγεθος? Πόσα byte ανά εντολή?
  - Πόσες διαφορετικές μορφές?

Τύποι δεδομένων σε μια ISA

- Κοινοί τύποι δεδομένων:
  - Integer
  - Floating-point (single- and double-precision)
  - Character
- Καθορίζει τον τρόπο με τον οποίο τα δεδομένα αναπαρίστανται και χειρίζονται από τον επεξεργαστή
- Μεγέθη:
  - 32-bit και 64-bit: Καθορίζει το μέγεθος των διευθύνσεων δεδομένων και μνήμης που μπορεί να χειριστεί ο επεξεργαστής
  - Υπάρχουν και άλλες εκδόσεις με περισσότερα ή λιγότερα bits

Registers σε μια ISA

- Αποθήκευση δεδομένων μέσα στον επεξεργαστή (core)
- Γενικής Χρήσης (General Purpose Registers (GPRs)):
  - Χρησιμοποιούνται για την προσωρινή αποθήκευση δεδομένων και τελεστών
- Program Counter (PC):
  - Δείχνει την επόμενη εντολή στο πρόγραμμα που θα εκτελεστεί
- Status Registers:
  - Stores flags (e.g., zero, carry, overflow)

Addressing Modes σε μια ISA

- Πως έχουμε πρόσβαση στα δεδομένα (operands)
- Κοινά addressing modes:
  - **Immediate**: Τα δεδομένα είναι μέρος της εντολής
  - **Register**: Τα δεδομένα είναι αποθηκευμένα σε ένα register
  - **Direct**: Τα δεδομένα είναι αποθηκευμένα στη μνήμη
  - **Indirect**: Μια διεύθυνση που δείχνει σε μια άλλα διεύθυνση στη μνήμη

Τύποι εντολών σε μια ISA

- Data Transfer Instructions:
  - Load and store operations between registers and memory
  - Example: `MOV, LDR`
- Arithmetic and Logical Instructions:
  - Add, subtract, multiply, bitwise operations
  - Example: `ADD, SUB, AND`
- Control Flow Instructions:
  - Branches, jumps, and conditionals
  - Example: `JMP, BEQ`
- Special Instructions:
  - System calls, input/output

Μεγέθη εντολών σε μια ISA

- **Σταθερού μεγέθους:**
  - Ευκολότερη αποκωδικοποίηση, ταχύτερη εκτέλεση
  - Παράδειγμα: Οι περισσότεροι επεξεργαστές RISC (ARM)
- **Μεταβλητού μεγέθους:**
  - Πιο ευέλικτη αλλά πολύπλοκη αποκωδικοποίηση
  - Παράδειγμα: Επεξεργαστές CISC (x86)

- Tradeoff: Speed vs Flexibility

ISA γενικές κατηγορίες: CISC & RISC

- Υπάρχουν πολλοί τρόποι να κατηγοριοποιήσουμε ISA
- Δυο κατηγορίες ομαδοποιούν όλα τα προηγούμενα:
  - **CISC**: Complex Instruction Set Computers
  - **RISC**: Reduced Instruction Set Computers
- Η RISC ήταν για πολλά χρόνια σε ερευνητικό επίπεδο (ARM, MIPS, IBM Power)
- Η CISC ήταν πιο διαδεδομένη σε προϊόντα (Intel, AMD)

Τι καθορίζει μια καλή/κακή ISA

- Ευκολία υλοποίησης στο επίπεδο του επεξεργαστή
  - Μπορεί να οδηγήσει σε αποδοτικές υλοποιήσεις (performance, power, cost)?
- Ευκολία προγραμματισμού στο επίπεδο του compiler
  - Μπορεί ο compiler να χρησιμοποιήσει μια ISA αποτελεσματικά?
- Συμβατότητα
  - Για πόσα χρόνια θα διαρκέσει?
  - Θα χρειαστεί να αλλάξει στα επόμενα 30 χρόνια?

CISC vs RISC

Complex Instruction Set Computers (CISC)

- Μεγάλο σύνολο εντολών
- Περίπλοκες εντολές
- Μικρότερα προγράμματα (πιο λίγες εντολές ανά πρόγραμμα)
- Μεταβλητό μέγεθος εντολών
- Πολλές διαφορετικές εντολές για την ίδια λειτουργία
- Πολύπλοκα μοντέλα διευθύνσεων
- Μεταβλητός αριθμός κύκλων εκτέλεσης για διαφορετικές εντολές
- Χρειάζονται πολλά transistor για να υλοποιήσουμε μια εντολή
- Πολυπλοκότητα → Υλικό (Hardware/CPU)
- "Πιο αργή" εκτέλεση ανά εντολή

Reduced Instruction Set Computers (RISC)

- Μικρό σύνολο εντολών
- Απλές εντολές
- Μεγαλύτερα προγράμματα (πιο πολλές εντολές ανά πρόγραμμα)
- Σταθερό μέγεθος εντολών
- Περιορισμένος αριθμός εντολών για την ίδια λειτουργία
- Απλά μοντέλα διευθύνσεων
- Σταθερός (συνήθως) αριθμός κύκλων εκτέλεσης για διαφορετικές εντολές
- Λίγα transistor ανά εντολή → Μπορούμε να βάλουμε περισσότερους Registers
- Πολυπλοκότητα → Λογισμικό (Compiler)
- "Πιο γρήγορη" εκτέλεση ανά εντολή

ISA Design Trade-offs

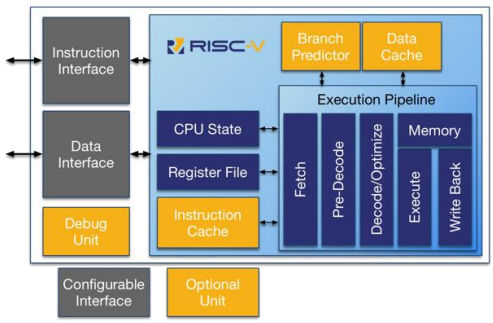
- Επίδοση και Πολυπλοκότητα
  - CISC πιο δύσκολο να βελτιστοποιήσουμε (ήδη βελτιστοποιημένη)
  - RISC πιο απλό hardware → πιο εύκολο να βελτιώσουμε (σαν προγραμματιστές) την επίδοση
- Απόδοση Ισχύς (Power efficiency)
  - RISC - ARM!! Champion of power efficiency
- Συμβατότητα
  - Περιορίζει την καινοτομία λόγω της ανάγκης να είναι συμβατή με legacy code/programs

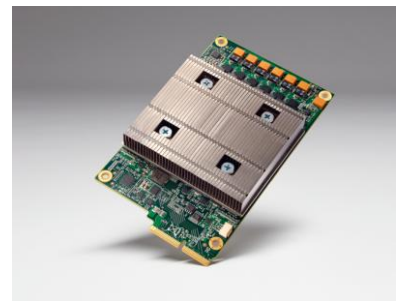
ISA Extensions

- Instruction Set Extensions
  - Ειδικευμένες εντολές σε συγκεκριμένες λειτουργίες που δεν υπήρχαν όταν δημιουργήθηκε μια ISA
  - Παραδείγματα: Vector extensions
    - MMX, SSE (for multimedia), NEON (for ARM).
- **Σκοπός:** Να βελτιώσουν την επίδοση συγκεκριμένων εφαρμογών όπως πολυμέσα/γραφικά και κρυπτογραφία

Το "Μέλλον" των ISA

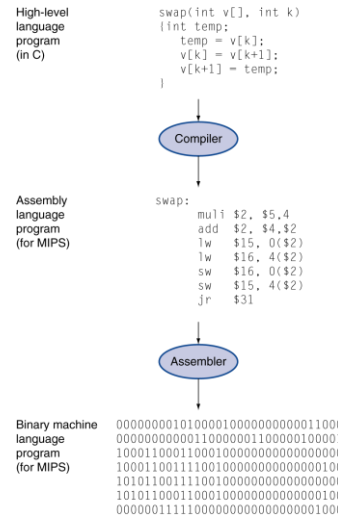
- RISC-V: Open-source ISA
- Custom-ISA: Accelerators with special purpose ISAs (e.g. Google TPU)



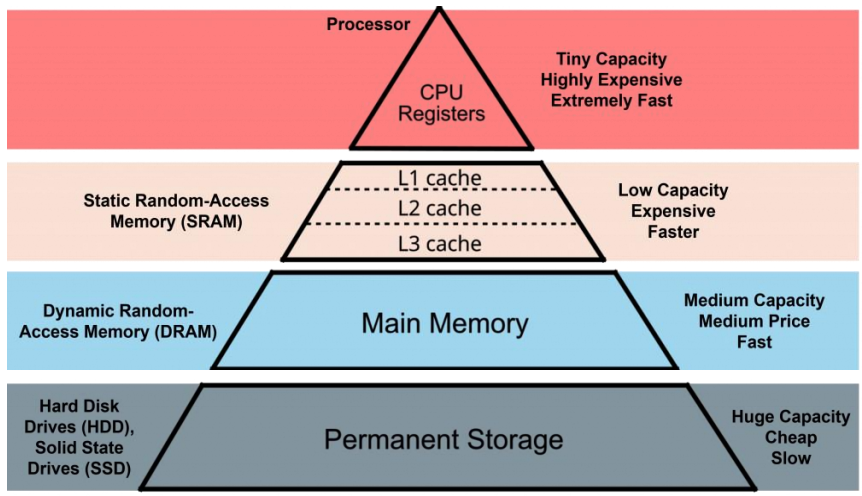


Προγραμματιστική Αφαιρετικότητα

- **High-level language**
  - Πιο κοντά στο φυσικό πρόβλημα που προσπαθούμε να λύσουμε
  - Μας παρέχει productivity (παραγωγικότητα) και portability
- **Assembly language**
  - Αναπαράσταση εντολών με κείμενο
- **Hardware representation**
  - Binary digits (bits)
  - Encoded instructions and data



Ιεραρχία της μνήμης



MIPS ISA

- Αναπτύχθηκε στο Πανεπιστήμιο του Stanford από τον Καθ. John Hennessy και μια ομάδα μεταπτυχιακών φοιτητών
- Παρόμοια με τις άλλες αρχιτεκτονικές μικροεπεξεργαστών που αναπτύχθηκαν από τη δεκαετία του 1980
- Το 1984 δημιουργήθηκε η MIPS Computer Systems Corp., για την κατασκευή υπολογιστών Unix υψηλής επίδοσης με MIPS μικροεπεξεργαστές
  - Σήμερα MIPS Technologies (www.mips.com)



Αρχές Σχεδιασμού ISA

1. «Η απλότητα ευνοεί την κανονικότητα»
   - Simplicity favors regularity
2. «Το μικρότερο είναι και το γρηγορότερο»
   - Smaller is faster
3. «Η συνήθης πράξη πρέπει να είναι γρήγορη»
   - Make the common case fast
4. «Ο καλός σχεδιασμός απαιτεί συμβιβασμό»
   - Good design requires compromise

Αριθμητική MIPS

- Όλες οι εντολές πρόσθεσης και αφαίρεσης έχουν 3 τελεστές (operands)
- Όλες οι αριθμητικές πράξεις στη MIPS έχουν την ίδια μορφή
- Η διάταξη των τελεστών είναι σταθερή (πρώτα ο τελεστής του αποτελέσματος)

Παράδειγμα:

Κώδικας C: `a = b + c;`

Κώδικας MIPS: `add $a, $b, $c`

a, b, c είναι καταχωρητές (registers)

Αριθμητική MIPS

- Η πρόσθεση 2 ή περισσότερων αριθμών εκτελείται από μία σειρά εντολών πρόσθεσης 2 αριθμών

Παράδειγμα:

Κώδικας C: `a = b + c + d + e;`

Κώδικας MIPS:

```
add $a, $b, $c      # a = b + c
add $a, $a, $d      # a = b + c + d
add $a, $a, $e      # a = b + c + d + e
```

- # -- το σύμβολο για τα σχόλια, ισχύει μέχρι το τέλος της γραμμής
- Η assembly επιτρέπει μόνο 1 εντολή ανά γραμμή

Αριθμητική MIPS

- **Οι τελεστές πρέπει να είναι καταχωρητές**, Όχι διευθύνσεις μνήμης
  - Σε αντίθεση με τους μικροεπεξεργαστές της Intel που επιτρέπουν σε αριθμητικές εντολές να έχουν κάποιους τελεστές στη μνήμη
- **1η Αρχή Σχεδιασμού:** «Η απλότητα ευνοεί την κανονικότητα»
  - Simplicity favors regularity
  - Απαιτεί μεγαλύτερο αριθμό εντολών για την επίλυση ενός προβλήματος
  - π.χ. χρειαζόμαστε **n** εντολές για την πρόσθεση **n+1** αριθμών
- Για την MIPS-32:
  - Έχουμε μόνο 32 καταχωρητές
  - Ο κάθε καταχωρητής είναι 32 bits

Αριθμητική MIPS

- Πιο είναι το αποτέλεσμα του μεταγλωττιστή για το πιο κάτω C πρόγραμμα;
  - Εντολές C → Μεταγλωττιστής C → Εντολές MIPS

```
a = b + c;   →   add $a, $b, $c
d = a - e;   →   sub $d, $a, $e
```

το ίδιο με…

```
a = b + c;   →   add $s1, $s2, $s3
d = a - e;   →   sub $s4, $s1, $s5
```

Αριθμητική MIPS

- Πιο είναι το αποτέλεσμα του μεταγλωττιστή για το πιο κάτω C πρόγραμμα;
  - Εντολές C → Μεταγλωττιστής C → Εντολές MIPS

```
f = (g + h) - (i + j);  →   add $t0, $g,  $h
                            add $t1, $i,  $j
                            sub $f,  $t0, $t1
```


Καταχωρητές (Registers)

- Χρησιμοποιούνται για δεδομένα που χρειάζονται συχνά στο πρόγραμμα
- Οι τελεστές αριθμητικών εντολών πρέπει να είναι καταχωρητές, παρέχονται μόνο 32 καταχωρητές των 32-bit

- **2η Αρχή Σχεδιασμού:** «Το μικρότερο είναι και το γρηγορότερο»
  - Smaller is faster

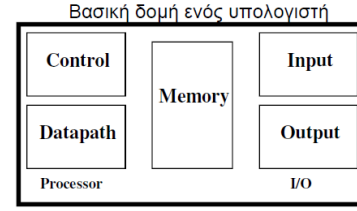
- Ονόματα καταχωρητών:
  - `$s0, $s1, $s2,…` : Καταχωρητές που αντιστοιχούν σε μεταβλητές στο πρόγραμμα υψηλού επιπέδου
  - `$t0, $t1, $t2,…` : για προσωρινές μεταβλητές απαραίτητες για το μεταγλωττισμό σε κώδικα MIPS

Καταχωρητές vs. Μνήμη

- Η καταχωρητές είναι ταχύτεροι από την μνήμη!
- Ο μεταγλωττιστής συσχετίζει μεταβλητές (variables) με καταχωρητές (registers)
- Τι συμβαίνει με προγράμματα που έχουν μεγάλο αριθμό μεταβλητών;
- Η μνήμη *(memory)* χρησιμοποιείται για αποθήκευση των εντολών, δεδομένων και αποτελεσμάτων ενός προγράμματος
- Τα δεδομένα μεταφέρονται από την μνήμη στους καταχωρητές
  - Χρειαζόμαστε εντολές για μεταφορά δεδομένων (*data transfer instructions) για φόρτωση από- και αποθήκευση προς τη μνήμη, όπως*

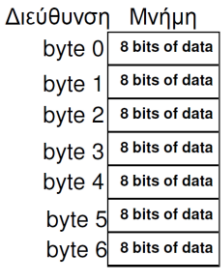
    `lw` → *load word*
    `sw` → *store word*

Βασική δομή ενός υπολογιστή



Οργάνωση Μνήμης – Εντολή Φόρτωσης/Load (`lw`)

- Η μνήμη θεωρείται ως ένας μεγάλος, μονοδιάστατος πίνακας λέξεων/words με συγκεκριμένη αρχική διεύθυνση (base address)
- Για να έχει πρόσβαση στη μνήμη μια εντολή πρέπει να μπορεί να παρέχει τη «διεύθυνση» των ζητούμενων δεδομένων
- Μια διεύθυνση μνήμης (memory address) καθορίζεται με ένα δείκτη στον πίνακα (0, 1, 2, …)
- «Διευθυνσιοδότηση με Byte» (Byte addressing) υπονοεί ότι ο δείκτης δείχνει σε ένα byte (= 8 συνεχόμενα bits) του πίνακα



Θεωρήστε ένα πίνακα A με 100 λέξεις (όπου 1 λέξη = 1 byte),
με g → $s1, h → $s2, και ο καταχωρητής αρχικής διεύθυνσης (*base address*) → $s3

```
g = h + A[5];
lw $t0, 5($s3)        # lw = load word
add $s1, $s2, $t0
```

base register (`$s3`): κρατά την αρχική διεύθυνση του πίνακα, δηλ. τη διεύθυνση του 1ου στοιχείου (λέξης) του πίνακα
Offset (`5`): ο δείκτης στον πίνακα, σταθερός στην εντολή

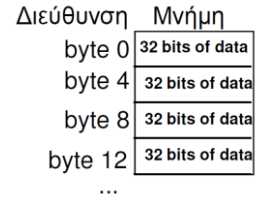
Διεύθυνση του στοιχείου = base register + offset

Οργάνωση Μνήμης για MIPS

- Συνήθως μια «λέξη» (word) είναι μεγαλύτερη από 1 byte
- Στη MIPS, οι καταχωρητές μπορούν να κρατήσουν μέχρι και 32 bits, άρα, μια λέξη έχει 32 bits ή 4 bytes

  - Byte address της 0ης λέξης = 0 (= 0 x 4)
  - Byte address της 1ης λέξης = 4 (= 1 x 4)
  - Byte address της 2ης λέξης = 8 (= 2 x 4)

  …
  - Byte address της nης λέξης = n x 4



- Έχουμε 2^{32} bytes σε byte addresses: 0, 1, 2, …, 2^{32}-1
- Έχουμε 2^{30} λέξεις σε byte addresses: 0, 4, 8, ..., 2^{32}-4 (***Big Endian***)
- Για το παράδειγμα της προηγούμενης διαφάνειας (`g = h + A[5];`):

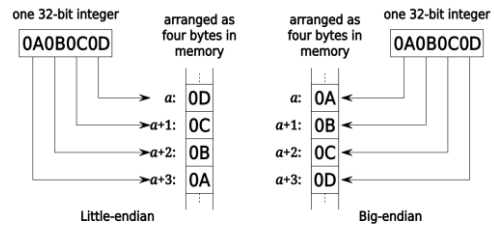
```
lw  $t0, 20($s3)     # byte address του offset είναι 5x4=20
add $s1, $s2, $t0
```


Big Endian vs Little Endian

- Καθορίζει την σειρά που διαβάζονται τα δεδομένα (ανά byte) σε μια λέξη/word

- **Big Endian**: Αριστερά → Δεξιά
- **Little Endian**: Δεξιά → Αριστερά

- *Σημαντικό κυρίως για επικοινωνία μεταξύ υπολογιστών!*



Εντολές Φόρτωσης και Αποθήκευσης - Παράδειγμα

Κώδικας C:
```
A[12] = h + A[8];
```

Κώδικας MIPS:
```
lw  $t0, 32($s3)          # 32 = 8x4
add $t0, $s2, $t0
sw  $t0, 48($s3)          # store word
```

- Ο μεταγλωττιστής συσχετίζει:
  - base address register σε **$s3** (έχει τη διεύθυνση της 1ης λέξης του πίνακα A)
  - μεταβλητή **h** στον καταχωρητή **$s2**
- Η εντολή **sw** έχει τον τελεστή αποτελέσματος τελευταίο
- Θυμηθείτε ότι για αριθμητικές εντολές, όλοι οι τελεστές πρέπει να είναι καταχωρητές, όχι διευθύνσεις μνήμης!

```
add 48($s3), $s2, 32($s3)   -- ΛΑΘΟΣ
```


Αριθμητικές Εντολές με Σταθερές -- Άμεσες Εντολές/Immediate Instructions

- Θεωρήστε ότι ο αριθμός 4 είναι μία σταθερά στη γλώσσα C και έχει αποθηκευτεί στην διεύθυνση της μνήμης *AddrConstant4*. Η αρχική διεύθυνση στη μνήμη που έχει τα δεδομένα του προγράμματος βρίσκεται στον καταχωρητή **$s1**
- Για να προσθέσουμε το 4 στον **$s3**:

```
lw   $t0, AddrConstant4($s1)     # $t0 = constant 4
add  $s3, $s3, $t0               # $s3 = $s3 + $t0
ή
addi $s3, $s3, 4                 # $s3 = $s3 + 4
```

- Η εντολή ***addi*** ονομάζεται *«άμεση εντολή»* (*immediate instruction*). Επιτρέπει ένας τελεστής να είναι σταθερά
- **3η Αρχή Σχεδιασμού:** «Η συνήθης πράξη πρέπει να είναι γρήγορη»
  - Make the common case fast

Αριθμητικές Εντολές με Σταθερές -- Άμεσες Εντολές/Immediate Instructions

- Δεν υπάρχει εντολή **subi**
  - Χρησιμοποιούμε την εντολή **addi** αφού η σταθερά μπορεί να είναι και αρνητικός αριθμός

```
addi $s3, $s3, -4                # $s3 = $s3 - 4
```


Ο καταχωρητής 0 (Constant Zero Register)

- Καταχωρητής του 0 ($zero) είναι η σταθερά 0
  - Δεν μπορεί να διαγραφεί ή να χρησιμοποιηθεί για να γράψουμε πάνω του
- Χρήσιμο για κοινές πράξεις
  - Π.χ., αντιγραφή δεδομένων μεταξύ καταχωρητών

```
add $t2, $s1, $zero              # $t2 = $s1
```

Τι είδαμε μέχρι τώρα

- MIPS
  - Η φόρτωση/αποθήκευση γίνεται με λέξεις
    - 1 λέξη μεταφέρεται από/προς τη μνήμη
  - Η διευθυνσιοδότηση γίνεται με bytes (byte addressing)
  - Αριθμητική μόνο με καταχωρητές!

Εντολή MIPS	Πράξη που εκτελείται
`add $s1, $s2, $s3`	`$s1 = $s2 + $s3`
`addi $s1, $s2, 100`	`$s1 = $s2 + 100`
`sub $s1, $s2, $s3`	`$s1 = $s2 – $s3`
`lw $s1, 100($s2)`	`$s1 = Memory[$s2+100]`
`sw $s1, 100($s2)`	`Memory[$s2+100] = $s1`

Γλώσσα Μηχανής – Αναπαράσταση Εντολών

- Οι εντολές κρατούνται στον υπολογιστή ως μια σειρά από ηλεκτρικά σήματα ψηλής ή χαμηλής τάσης → δυαδικά ψηφία
- Οι εντολές, όπως και οι καταχωρητές και οι λέξεις δεδομένων, αποτελούνται από 32 bits
- Κάθε καταχωρητής έχει ένα αριθμό, από 0 σε 31
  - Π.χ. `$t1=9, $s1=17, $s2=18` (θα δούμε ολοκληρωμένη λίστα σύντομα)

Γλώσσα Μηχανής – Αναπαράσταση Εντολών

- Μορφή Εντολής (Instruction Format):
  - Καθορίζεται από τη διάταξη της εντολής (των bits)

op	rs	rt	rd	shamt	funct	Διάταξη Αριθμητικής Εντολής
0	17	18	9	0	32	Γλώσσα Μηχανής (Δεκαδικό)
000000	10001	10010	01001	00000	100000	Γλώσσα Μηχανής (Δυαδικό)

- **Παράδειγμα:** Η αριθμητική εντολή: `add $s0, $s1, $s2`

Γλώσσα Μηχανής – Αναπαράσταση Εντολών

- Για λόγους απλότητας και αναγνωσιμότητας, οι εντολές της μηχανής γράφονται στο δεκαεξαδικό (Hex) σύστημα, αντί στο δυαδικό

- Η εντολή

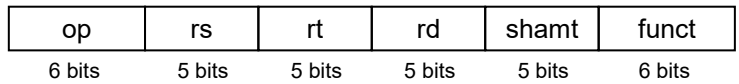
$$(000000\ 10001\ 10010\ 01001\ 00000\ 100000)_2$$

- γίνεται

$$(02324820)_{16}$$

MIPS R-format Instructions

- R-format: Αριθμητικές εντολές
- Κάθε πεδίο έχει αμετάβλητο μέγεθος και παρέχει στον υπολογιστή συγκεκριμένη πληροφορία



- Πεδία εντολών:
  - op: βασικός κωδικός εντολής (opcode)
  - rs: καταχωρητής για 1ο τελεστή (1st source register)
  - rt: καταχωρητής για 2ο τελεστή (2nd source register)
  - rd: καταχωρητής αποτελέσματος (destination register)
  - shamt: shift amount (00000 for now)
  - funct: συγκεκριμένη αριθμητική λειτουργία (function code -- extends opcode)

MIPS R-format - Παράδειγμα

op	rs	rt	rd	shamt	funct
6 bits	5 bits	5 bits	5 bits	5 bits	6 bits

```
add $t0, $s1, $s2
```

special	$s1	$s2	$t0	0	add
0	17	18	8	0	32
000000	10001	10010	01000	00000	100000

$$00000010001100100100000000100000_2 = 02324020_{16}$$

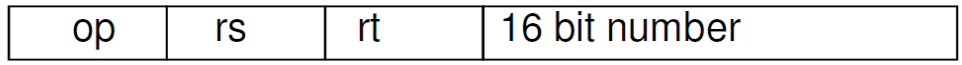
MIPS I-format Instructions

op	rs	rt	constant or address
6 bits	5 bits	5 bits	16 bits

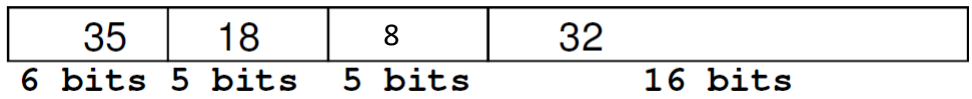
- Για εντολές που έχουν να κάνουν με immediate (σταθερές) τιμές
  - Load word (`lw`), Store word (`sw`), Immediate addition (`addi`), κ.α.
  - Εντολές που χρειάζονται μόνο 2 καταχωρητές
    - rt: Καταχωρητής αποτελέσματος (destination register)
    - rs: Καταχωρητής 1ου τελεστή (source register)
    - Constant: τιμές από -2^{15} μέχρι $+2^{15} - 1$
    - Address: offset που προσθέτουμε στο *base address* στο `rs`

- **4η Αρχή Σχεδιασμού:** «Ο καλός σχεδιασμός απαιτεί συμβιβασμό»
  - Good design requires compromise
  - Διαφορετικές μορφές περιπλέκουν την αποκωδικοποίηση
  - Διατηρούμε τις μορφές όσο το δυνατόν παρόμοιες (π.χ. όλες είναι 32-bit)

MIPS I-format - Παράδειγμα




```
lw $t0, 32($s2)
```



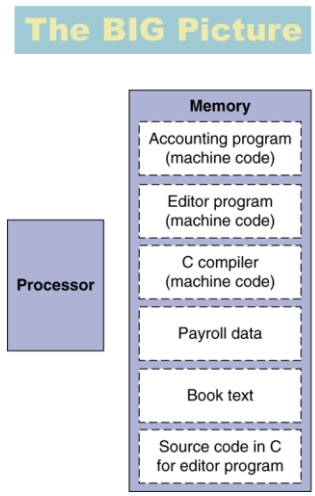
Αριθμητική MIPS

MIPS machine language

Name	Format	Example						Comments
add	R	0	18	19	17	0	32	add $s1,$s2,$s3
sub	R	0	18	19	17	0	34	sub $s1,$s2,$s3
addi	I	8	18	17	100			addi $s1,$s2,100
lw	I	35	18	17	100			lw $s1,100($s2)
sw	I	43	18	17	100			sw $s1,100($s2)
Field size		6 bits	5 bits	5 bits	5 bits	5 bits	6 bits	All MIPS instructions 32 bits
R-format	R	op	rs	rt	rd	shamt	funct	Arithmetic instruction format
I-format	I	op	rs	rt	address			Data transfer format

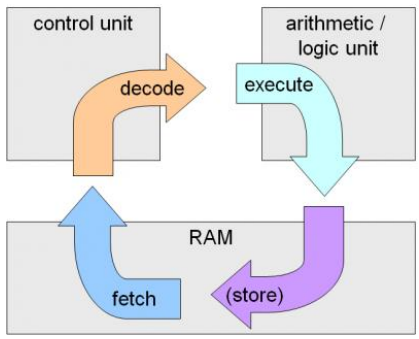
- Οι εντολές **`add`** και **`sub`** έχουν το ίδιο πεδίο op
- Το πεδίο **`funct`** καθορίζει την αριθμητική εντολή (`sub, add`) που θα εκτελεστεί

Η έννοια του Αποθηκευμένου Προγράμματος



- Οι εντολές αναπαρίστανται σαν bits, όπως και τα δεδομένα
- Όπως και τα δεδομένα, οι εντολές αποθηκεύονται στη μνήμη του υπολογιστή
- Ένα πρόγραμμα μπορεί να είναι τα δεδομένα για κάποιο άλλο πρόγραμμα
  - Π.χ. Compilers, linkers

Fetch and Execute Cycle



- Κύκλος Προσκόμισης και Εκτέλεσης
  - Η κάθε εντολή προσκομίζεται (fetched) και τοποθετείται σε ειδικό καταχωρητή (Instruction Register - IR)
  - Η εντολή αποκωδικοποιείται (decoded) για να καταλάβει ο επεξεργαστής τι εντολή είναι
  - Ανατίθεται σε μια αριθμητική μονάδα εκτέλεσης (ALU) για εκτέλεση
  - Τα αποτελέσματα γράφονται πίσω στη μνήμη/registers (και στα δύο)
- **Παράλληλα** προσκομίζεται η «επόμενη» εντολή στο πρόγραμμα
  - Ο Program Counter (PC) είναι ειδικός καταχωρητής που διατηρεί την διεύθυνση της επόμενης εντολής προς εκτέλεση

Λογικές Εντολές MIPS (Logic Instructions)

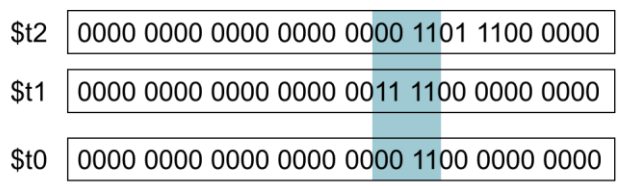
Logical operations	C operators	Java operators	MIPS instructions
Shift left	<<	<<	sll
Shift right	>>	>>>	srl
Bit-by-bit AND	&	&	and, andi
Bit-by-bit OR	|	|	or, ori
Bit-by-bit NOT	~	~	nor

- Για λειτουργίες shift (**sll, srl**), συμπληρώνουμε με μηδέν
- **and/or/not** είναι πράξεις $3^{ων}$ τελεστών/καταχωρητών, π.χ.
  - `and $t0, $t1, $t2  # $t0 = $t1 & $t2`
  - `nor $t0, $t1, $t2  # $t0 = ~( $t1 | $t2 )`
- Ποιος τύπος διάταξης εντολής μπορεί να χρησιμοποιηθεί για αυτές τις εντολές λογικής (I-type of R-type);
  - R-type

Εντολές AND

- Bit-by-bit AND !!! (όχι το λογικό and της C/Java)
- Χρήσιμο για να κάνουμε mask κάποια bits σε μια λέξη/word
  - Επιλέγουμε κάποια bits και τα υπόλοιπα γίνονται 0

`and $t0, $t1, $t2`



Εντολές OR

- Bit-by-bit OR !!! (όχι το λογικό or της C/Java)
- Χρήσιμο για να συμπεριλάβουμε κάποια bits σε μια λέξη/word
  - Ορίζουμε κάποια bits στην τιμή 1 και τα υπόλοιπα δεν αλλάζουν

```
or $t0, $t1, $t2
```

$t2	0000 0000 0000 0000 0000 1101 1100 0000
$t1	0000 0000 0000 0000 0011 1100 0000 0000
$t0	0000 0000 0000 0000 0011 1101 1100 0000

Εντολές NOT

- Bit-by-bit NOT !!! (όχι το λογικό not της C/Java)
- Χρήσιμο για να αντιστρέψουμε τα bits σε μια λέξη/word
  - Αλλάζει τα 0 σε 1 και τα 1 σε 0
- Στη MIPS το NOT αναπαριστάτε με 3 τελεστών NOR εντολή
  - `a NOR b == NOT ( a OR b )`

```
nor $t0, $t1, $zero
```

Register 0: always read as zero

$t1	0000 0000 0000 0000 0011 1100 0000 0000
$t0	1111 1111 1111 1111 1100 0011 1111 1111

Λογικές Εντολές Immediate

- Υπάρχουν επίσης και οι εντολές **andi** και **ori** για άμεσες εντολές λογικής με σταθερό
  - `andi $s1, $s2, 100  # $s1 = $s2 & 100`
  - `ori $s1, $s2, 100  # $s1 = $s2 | 100`
- Ποιος τύπος διάταξης εντολής μπορεί να χρησιμοποιηθεί για αυτές τις εντολές λογικής (I-type of R-type);
  - I-type

Εντολές Shift

op	rs	rt	rd	shamt	funct
6 bits	5 bits	5 bits	5 bits	5 bits	6 bits

```
sll $t2, $s0, 4          # $t2 = $s0 << 4
```

- Το περιεχόμενο του `$s0` μετακινείται 4 bits αριστερά, τα 4 bits στα δεξιά παίρνουν την τιμή 0 και το αποτέλεσμα αποθηκεύεται στον `$t2`
- Το 4 ονομάζεται το «ποσό ολίσθησης» (shift amount) και δηλώνεται στο πεδίο shamt της διάταξης εντολής R-type
- Shift left logical
  - **sll** by i bits multiplies by 2^i
- Shift right logical
  - **srl** by i bits divides by 2^i

Εντολές Ελέγχου (Control Instructions)

- Εντολές Λήψεως Αποφάσεων ή Εντολές Ελέγχου
  - Αλλάζουν την ροή εκτέλεσης του προγράμματος
  - Απαραίτητες αφού η σειρά εκτέλεσης των εντολών μπορεί να εξαρτάται από τις τιμές δεδομένων που πιθανόν να αλλάζουν κατά τη διάρκεια ενός προγράμματος
  - Δηλ., καθορίζουν την «επόμενη» εντολή προς εκτέλεση
- Εντολές MIPS με διακλάδωση υπό συνθήκη (conditional branch):
  - `bne $t0, $t1, L1` # Εάν `$t0 ≠ $t1` η επόμενη εντολή βρίσκεται στη διεύθυνση L1
  - `beq $t0, $t1, L1` # Εάν `$t0 == $t1` η επόμενη εντολή βρίσκεται στη διεύθυνση L1

Εντολές Ελέγχου (Control Instructions)

- Εντολές MIPS με διακλάδωση χωρίς συνθήκη (unconditional branch):
  - `j Label`
- Ονομάζεται και εντολή άλματος (*jump instruction)*

Παράδειγμα If Statements

C Code:

```
if (i == j)
   h = i + j;
```

#(i→$s0, j→$s1, h→$s3)

MIPS:

```
bne $s0, $s1, L1
add $s3, $s0, $s1
L1: ....
```

~~beq $s0, $s1, L1~~

```
L1: add $s3, $s0, $s1
```

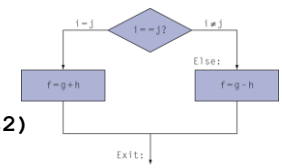

Παράδειγμα If Statements

C Code:

```
if (i == j)
   f = g + h;
else
   f = g - h;
```

#(i→$s3, j→$s4)

#(f→$s0, g→$s1, h→$s2)



MIPS:

```
        bne $s3, $s4, Else
        add $s0, $s1, $s2
        j   Exit
Else:   sub $s0, $s1, $s2
Exit:   …
```

```
  beq $s3, $s4, If
  sub $s0, $s1, $s2
  j   Exit

If:  add $s0, $s1, $s2
Exit: …
```

Άσκηση για το σπίτι: Loop Statements

C Code: `while (save[i] == k) i += 1;` `#(i→$s3, k→$s5, base address of save[] in $s6)`

MIPS: `?????`

61

MIPS operands

Name	Example	Comments
32 registers	`$s0, $s1, . . . , $s7` `$t0, $t1, . . . , $t7,` `$zero`	Fast locations for data. In MIPS, data must be in registers to perform arithmetic. Registers `$s0`–`$s7` map to 16–23 and `$t0`–`$t7` map to 8–15. MIPS register $zero always equals 0.
2^{30} memory words	Memory[0], Memory[4], . . . , Memory[4294967292]	Accessed only by data transfer instructions in MIPS. MIPS uses byte addresses, so sequential word addresses differ by 4. Memory holds data structures, arrays, and spilled registers.

62

MIPS assembly language

Category	Instruction	Example	Meaning	Comments
Arithmetic	add	`add $s1,$s2,$s3`	$s1 = $s2 + $s3	Three operands; data in registers
	subtract	`sub $s1,$s2,$s3`	$s1 = $s2 – $s3	Three operands; data in registers
Data transfer	load word	`lw $s1,100($s2)`	$s1 = Memory[$s2 + 100]	Data from memory to register
	store word	`sw $s1,100($s2)`	Memory[$s2 + 100] = $s1	Data from register to memory
Logical	and	`and $s1,$s2,$s3`	$s1 = $s2 & $s3	Three reg. operands; bit-by-bit AND
	or	`or $s1,$s2,$s3`	$s1 = $s2 | $s3	Three reg. operands; bit-by-bit OR
	nor	`nor $s1,$s2,$s3`	$s1 = ~ ($s2 |$s3)	Three reg. operands; bit-by-bit NOR
	and immediate	`andi $s1,$s2,100`	$s1 = $s2 & 100	Bit-by-bit AND reg with constant
	or immediate	`ori $s1,$s2,100`	$s1 = $s2 | 100	Bit-by-bit OR reg with constant
	shift left logical	`sll $s1,$s2,10`	$s1 = $s2 << 10	Shift left by constant
	shift right logical	`srl $$s1,$s2,10`	$s1 = $s2 >> 10	Shift right by constant
Conditional branch	branch on equal	`beq $s1,$s2,L`	if ($s1 == $s2) go to L	Equal test and branch
	branch on not equal	`bne $s1,$s2,L`	if ($s1 != $s2) go to L	Not equal test and branch
	set on less than	`slt $s1,$s2,$s3`	if ($s2 < $s3) $s1 = 1; else $s1 = 0	Compare less than; used with `beq, bne`
	set on less than immediate	`slt $s1,$s2,100`	if ($s2 < 100) $s1 = 1; else $s1 = 0	Compare less than immediate; used with `beq, bne`
Unconditional jump	jump	`j L`	go to L	Jump to target address

63

MIPS machine language

Name	Format	Example						Comments
`add`	R	0	18	19	17	0	32	`add $s1,$s2,$s3`
`sub`	R	0	18	19	17	0	34	`sub $s1,$s2,$s3`
`lw`	I	35	18	17	100			`lw $s1,100($s2)`
`sw`	I	43	18	17	100			`sw $s1,100($s2)`
`and`	R	0	18	19	17	0	36	`and $s1,$s2,$s3`
`or`	R	0	18	19	17	0	37	`or $s1,$s2,$s3`
`nor`	R	0	18	19	17	0	39	`nor $s1,$s2,$s3`
`andi`	I	12	18	17	100			`andi $s1,$s2,100`
`ori`	I	13	18	17	100			`ori $s1,$s2,100`
`sll`	R	0	0	18	17	10	0	`sll $s1,$s2,10`
`srl`	R	0	0	18	17	10	2	`srl $s1,$s2,10`
`beq`	I	4	17	18	25			`beq $s1,$s2,100`
`bne`	I	5	17	18	25			`bne $s1,$s2,100`
`slt`	R	0	18	19	17	0	42	`slt $s1,$s2,$s3`
`j`	J	2	2500					`j 10000` (see Section 2.9)
Field size		6 bits	5 bits	5 bits	5 bits	5 bits	6 bits	All MIPS instructions 32 bits
R-format	R	op	rs	rt	rd	shamt	funct	Arithmetic instruction format
I-format	I	op	rs	rt	address			Data transfer, branch format

64

Τι είδαμε μέχρι τώρα

Εντολή	**Πράξη που εκτελείται**
`add $s1,$s2,$s3`	$s1 = $s2 + $s3
`sub $s1,$s2,$s3`	$s1 = $s2 – $s3
`lw $s1,100($s2)`	$s1 = Memory[$s2+100]
`sw $s1,100($s2)`	Memory[$s2+100] = $s1
`bne $s4, $s5, L`	Επόμενη εντολή στο L εάν $s4 ≠ $s5
`beq $s4, $s5, L`	Επόμενη εντολή στο L εάν $s4 = $s5
`j Label`	Επόμενη εντολή στο Label

Εντολές Λογικής (`and, or, nor, sll, srl`)

Άμεσες Εντολές (`addi, andi, ori`)

Διατάξεις Εντολών:

R	op	rs	rt	rd	shamt	funct
I	op	rs	rt	16 bit address		
J	op	26 bit address				

Παράδειγμα (Λύση) Loop Statements

C Code:

```
while (save[i] == k) i += 1;    #(i→$s3, k→$s5, base address
                                   of save[] in $s6)
```

MIPS:

```
Loop: sll  $t1, $s3, 2     # $t1 = 4*i
      add  $t1, $t1, $s6   # $t1 = address of save[i]
      lw   $t0, 0($t1)     # $t0 = save[i]
      bne  $t0, $s5, Exit  # go to Exit if save[i] ≠ k
      addi $s3, $s3, 1     # i++
      j    Loop            # Jump back to the start of the loop
Exit: …
```


Εντολές Ελέγχου (Control Instructions)

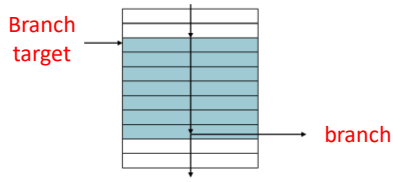
- **Set On Less than**:
  - `slt $t0, $s1, $s2` # if ($s1 < $s2) τότε $t0 = 1, αλλιώς $t0 = 0
  - `slti $t0, $s2, 10` # if ($s2 < 10) τότε $t0 = 1, αλλιώς $t0 = 0
- Μπορεί να χρησιμοποιηθεί σε συνδυασμό με τις **`beq`** και **`bne`**

```
slt $t0, $s1, $s2     # if ($s1 < $s2)
bne $t0, $zero, L     #   branch to L
```

- Γιατί δεν έχουμε απευθείας branch-if-less-than (blt, bge,…) εντολή?
  - Η υλοποίηση σε hardware των <, ≥, … είναι πιο αργή παρά η υλοποίηση των =, ≠
  - Θα χρειαζόταν να μεγαλώσουμε τον κύκλο (→ πιο αργός) της μηχανής
  - Και θα κάναμε ΟΛΕΣ τις εντολές της μηχανής πιο αργές!
  - Συμβιβασμός! (**4η Αρχή Σχεδιασμού**)

Basic Blocks

- Είναι μια ακολουθία από εντολές χωρίς:
  - Κανένα branch (εχτός από το τέλος)
  - Κανένα branch target (εχτός από την αρχή)




```
w = 0;
x = x + y;
y = 0;
if( x > z)
  {
   y = x;
   x++;
  }
else
  {
   y = z;
   z++;
  }
w = x + z;
```

Source Code

```
w = 0;
x = x + y;
y = 0;
if( x > z)
```

```
y = x;
x++;
```

```
y = z;
z++;
```

```
w = x + z;
```

Basic Blocks

- Ομαδοποιείται ο κώδικας σε basic blocks από τον compiler
  - Για ευκολία ανάγνωσης και βελτιστοποιήσεων

Υποστήριξη Συναρτήσεων

- **Συνάρτηση** (Procedure ή Function ή Routine):
  - Αποθηκευμένη μονάδα προγράμματος που υλοποιεί συγκεκριμένη εργασία βάση των παραμέτρων με τους οποίους καλείται

- Πλεονεκτήματα χρήσης συναρτήσεων:
  - Καλύτερη δομή προγράμματος
  - Αυξάνει την αναγνωσιμότητα του προγράμματος
  - Επαναχρησιμοποίηση!

Υποστήριξη Συναρτήσεων

- Τα 6 βήματα που πάντοτε ακολουθούνται για την εκτέλεση μιας συνάρτησης:
  1. Αποθήκευση παραμέτρων σε χώρο (registers) που έχει πρόσβαση η συνάρτηση
  2. Μεταφορά ελέγχου (εκτέλεσης) στη συνάρτηση
  3. Δέσμευση μνήμης που απαιτείται για την εκτέλεση της συνάρτησης
  4. Εκτέλεση των εντολών της συνάρτησης
  5. Αποθήκευση αποτελεσμάτων σε χώρο (register) που έχει πρόσβαση το κύριο πρόγραμμα (calling program)
  6. Επιστροφή ελέγχου/εκτέλεσης στο σημείο που καλέστηκε η συνάρτηση

Καταχωρητές για κλήση συναρτήσεων

- `$a0 – $a3`: παράμετροι προς τη συνάρτηση
- `$v0, $v1`: αποτελέσματα προς το κύριος πρόγραμμα
- `$sp`: stack pointer
- `$ra`: διεύθυνση επιστροφής

- `t0 – $t9`: temporaries
  - Μπορεί να τις διαγράψει μια καλούμενη συνάρτηση
- `$s0 – $s7`: saved
  - Πρέπει να αποθηκευτούν/επαναφορτωθούν στη μνήμη από την καλούμενη συνάρτηση

- Εντολή Κλήσης Συνάρτησης:

```
jal ProcedureAddress
```

  - εντολή jump-and-link: προχωρεί στη διεύθυνση της συνάρτησης ενώ συγχρόνως αποθηκεύει τη διεύθυνση της εντολής που ακολουθεί τη συνάρτηση στον ειδικό καταχωρητή `$ra`

Υποστήριξη Συναρτήσεων

- Η έννοια του αποθηκευμένου προγράμματος υπονοεί ότι η διεύθυνση της εντολής που εκτελείται είναι πάντα γνωστή
  - Φυλάγεται σε ένα καταχωρητή που ονομάζεται Μετρητής Προγράμματος (Program Counter -- PC)
  - Η εντολή `jal` αποθηκεύει PC + 4 στον `$ra`
- Πως επιστρέφουμε στο κύριο πρόγραμμα από μια συνάρτηση?

```
jr $ra          # Jump Register - άλμα (χωρίς συνθήκη) στη διεύθυνση που
                  κρατά ο καταχωρητής $ra
```

  - Αντιγράφει τα περιεχόμενα του `$ra` στον Program Counter (PC)

Υποστήριξη Συναρτήσεων

- Τι συμβαίνει αν η συνάρτηση απαιτεί περισσότερους καταχωρητές;
  - Το περιεχόμενο των καταχωρητών που χρησιμοποιούνται από τη συνάρτηση αποθηκεύονται πριν αυτή καλεστεί, για να μπορούν να επαναφερθούν όταν η συνάρτηση τερματίσει/επιστρέψει
  - Η αποθήκευση του περιεχόμενου καταχωρητών στη μνήμη ονομάζεται memory (ή register) spilling
  - Χρησιμοποιούμε τη στοίβα (stack) = δομή δεδομένων για αποθήκευση καταχωρητών, μέρος της μνήμης, δομή LIFO
  - Δείκτης Στοίβας (Stack Pointer -- `$sp`) δείχνει στην πιο πρόσφατα καταχωρημένη διεύθυνση στη στοίβα (most recently allocated address in the stack)
  - Χρησιμοποιούμε pop και push λειτουργίες για στοίβες

Παράδειγμα

- C code:

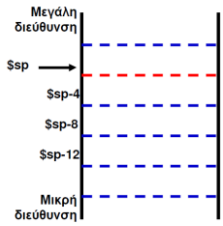
```
int leaf_example (int g, h, i, j)
{ int f;
  f = (g + h) - (i + j);
  return f;
}
```

- Παράμετροι g, …, j στους καταχωρητές $a0, …, $a3
- f στον $s0
- Αποτέλεσμα στον $v0

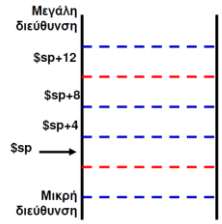
Παράδειγμα – Λύση 1

```
Leaf_example:                    # αρχή της συνάρτησης
addi $sp, $sp, -12               # κρατά χώρο στη στοίβα για 3 δεδομένα
sw $t1, 8($sp)                   # φύλαξη (push) $t1 στη στοίβα
sw $t0, 4($sp)                   # φύλαξη (push) $t0 στη στοίβα
sw $s0, 0($sp)                   # φύλαξη (push) $s0 στη στοίβα
```

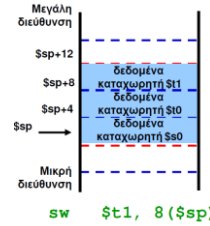
Register spilling στη στοίβα (push)



Αρχικά η στοίβα είναι άδεια




```
addi $sp, $sp, -12
```




```
sw   $t1, 8($sp)
sw   $t0, 4($sp)
sw   $s0, 0($sp)
```

Παράδειγμα – Λύση 1

Κύριος Μέρος της συνάρτησης

```
add $t0, $a0, $a1          # $t0 = g + h
add $t1, $a2, $a3          # $t1 = i + j
sub $s0, $t0, $t1          # f = (g + h) – (i + j)
```

Αποθήκευση τιμής αποτελέσματος

```
add $v0, $s0, $zero        # επιστροφή αποτελέσματος στον
                           # $v0 = f = $s0 + 0
```

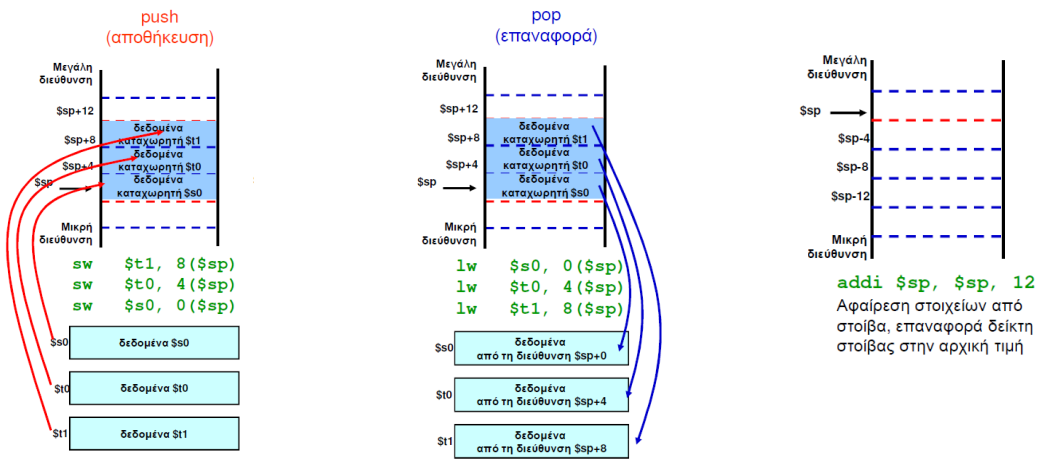
Επαναφορά καταχωρητών (pop)

```
lw $s0, 0($sp)             # επαναφορά $s0 από στοίβα
lw $t0, 4($sp)             # επαναφορά $t0 από στοίβα
lw $t1, 8($sp)             # επαναφορά $t1 από στοίβα
addi $sp, $sp, 12          # αφαίρεση 3 στοιχείων από την στοίβα
```

Επιστροφή

```
jr $ra                     # επιστροφή = jump πίσω στο κυρ. πρόγραμμα
```


Παράδειγμα – Λύση 1



Υποστήριξη Συναρτήσεων

- Πως γνωρίζουμε ποιος καταχωρητής πρέπει να φυλαχθεί πριν την εκτέλεση μιας συνάρτησης;

- Στη MIPS, οι καταχωρητές χωρίζονται σε 2 κατηγορίες:
  - `$t0 - $t9`: προσωρινοί (temporary) καταχωρητές, δεν διατηρούνται
  - `$s0 - $s7`: αποθηκευμένοι (saved) καταχωρητές, πρέπει να διατηρηθούν αν χρησιμοποιούνται από συνάρτηση

Παράδειγμα – Λύση 2

- MIPS code:

```
leaf_example:
        addi $sp, $sp, -4
        sw   $s0, 0($sp)
        add  $t0, $a0, $a1
        add  $t1, $a2, $a3
        sub  $s0, $t0, $t1
        add  $v0, $s0, $zero
        lw   $s0, 0($sp)
        addi $sp, $sp, 4
        jr   $ra
```

Save $s0 on stack

Procedure body

Result

Restore $s0

Return

Υποστήριξη Φωλιασμένων Συναρτήσεων

- Συναρτήσεις που καλούν άλλη συνάρτηση ή τον εαυτό τους (αναδρομή)
- Παράδειγμα:
  - **Συνάρτηση Α** με παράμετρο 5 (`$a0 = 5, $ra` αναθέτεται) καλείται από το κύριο πρόγραμμα
  - **Συνάρτηση Β** με παράμετρο 10 (`$a0 = 10, $ra` αναθέτεται ξανά) καλείται από την συνάρτηση Α
    - Μπορεί η Α να συνεχίσει με έγκυρα δεδομένα; ΟΧΙ
    - Μπορεί η Α να επιστρέψει στο κύριο πρόγραμμα;
- Πρέπει σε κάθε συνάρτηση να αποθηκεύουμε στη μνήμη
  - Τη διεύθυνση επιστροφής (`$ra`)
  - Άλλους καταχωρητές που κρατούν δεδομένα που θα χρειαστούμε κατά την επιστροφή
  - Όλα αυτά τα αποθηκεύουμε στο **Stack**

Υποστήριξη Φωλιασμένων Συναρτήσεων

C code:

```
int fact (int n)
{
  if (n < 1) return 1;
  else return n * fact(n - 1);
}
```

- n → **$a0**
- Αποτέλεσμα → **$v0**

Υποστήριξη Φωλιασμένων Συναρτήσεων

MIPS code:

```
fact:
    addi $sp, $sp, -8      # adjust stack for 2 items
    sw   $ra, 4($sp)       # save return address
    sw   $a0, 0($sp)       # save argument
    slti $t0, $a0, 1       # test for n < 1
    beq  $t0, $zero, L1
    addi $v0, $zero, 1     # if so, result is 1
    addi $sp, $sp, 8       #   pop 2 items from stack
    jr   $ra               #   and return
L1: addi $a0, $a0, -1      # else decrement n
    jal  fact              # recursive call
    lw   $a0, 0($sp)       # restore original n
    lw   $ra, 4($sp)       #   and return address
    addi $sp, $sp, 8       # pop 2 items from stack
    mul  $v0, $a0, $v0     # multiply to get result
    jr   $ra               # and return
```


Υποστήριξη Συναρτήσεων

- Στατικά (Static) Vs Αυτόματα (Automatic) δεδομένα:
  - Θυμηθείτε την κατηγορία μεταβλητών στατικής μνήμης (static storage class) στη C
    - οι μεταβλητές ισχύουν για ολόκληρο το πρόγραμμα (συμπεριλαμβανομένων όλων των συναρτήσεων) – κοινές (public) μεταβλητές
  - Σε αντίθεση, η κατηγορία μεταβλητών αυτόματης μνήμης (automatic storage class):
    - οι μεταβλητές ισχύουν μόνο μέσα στη συνάρτηση από την οποία έχουν οριστεί – τοπικές (local) μεταβλητές

- Η MIPS ορίζει τον καταχωρητή `$gp` ως γενικό δείκτη (global pointer) ο οποίος χρησιμοποιείται για να δείχνει στο κομμάτι της μνήμης που περιέχει τα στατικά δεδομένα/μεταβλητές

MIPS register conventions

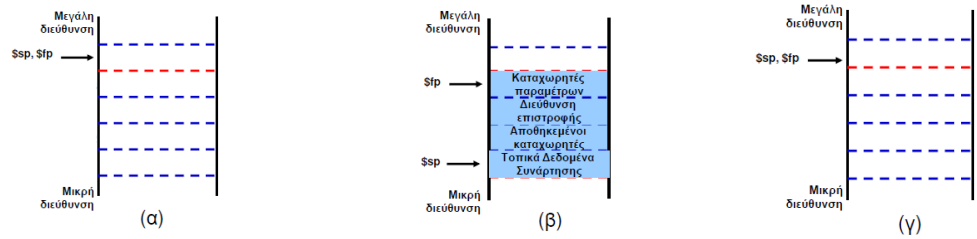
Name	Register number	Usage	Preserved on call?
$zero	0	the constant value 0	n.a.
$v0–$v1	2–3	values for results and expression evaluation	no
$a0–$a3	4–7	arguments	no
$t0–$t7	8–15	temporaries	no
$s0–$s7	16–23	saved	yes
$t8–$t9	24–25	more temporaries	no
$gp	28	global pointer	yes
$sp	29	stack pointer	yes
$fp	30	frame pointer	yes
$ra	31	return address	yes

- Ο καταχωρητής 1 (**`$at`**) χρησιμοποιείται αποκλειστικά από τον assembler
- Οι καταχωρητές 26-27 (**`$k0, $k1`**) χρησιμοποιούνται αποκλειστικά από το λειτουργικό σύστημα

Υποστήριξη Συναρτήσεων

- Κατανομή μνήμης στη στοίβα για επιπρόσθετα δεδομένα από συναρτήσεις:
  - Η στοίβα μπορεί να χρησιμοποιηθεί και για αποθήκευση τοπικών δεδομένων/μεταβλητών μιας συνάρτησης. Αυτό συμβαίνει όταν τα τοπικά δεδομένα/μεταβλητές μιας συνάρτησης δεν μπορούν όλα να αποθηκευτούν σε καταχωρητές (π.χ. πίνακες)
  - Πλαίσιο Συνάρτησης (procedure frame ή activation record): κομμάτι της στοίβας που περιέχει τους αποθηκευμένους καταχωρητές και τα τοπικά δεδομένα μιας συνάρτησης
  - Frame pointer -- **`$fp`**: ο καταχωρητής που δείχνει στο πλαίσιο μιας συνάρτησης (δηλ., έχει την αρχική διεύθυνση που δείχνει στην 1η λέξη του πλαισίου μιας συνάρτησης)

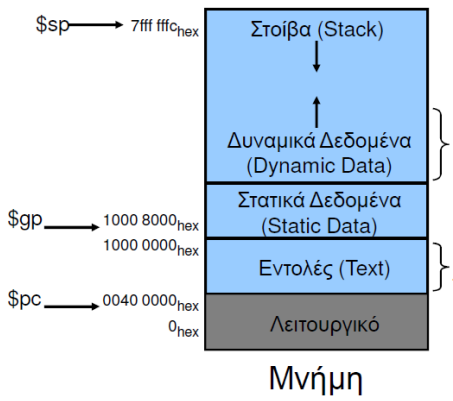
Υποστήριξη Συναρτήσεων



- Κατανομή στοίβας (α) πριν, (β) μετά, και (γ) κατά τη διάρκεια κλήσεως συνάρτησης:
  - `$fp` δείχνει στην 1η λέξη του πλαισίου της συνάρτησης
  - `$sp` δείχνει στην αρχή της στοίβας
  - `$sp` αλλάζει κατά τη διάρκεια εκτέλεσης της συνάρτησης ενώ ο `$fp` είναι αμετάβλητος, άρα, για πρόσβαση στα δεδομένα χρησιμοποιείται ο `$fp`
  - `$fp` αρχικοποιείται με την τιμή του `$sp` και η τιμή του `$sp` επαναφέρεται στο τέλος από τον `$fp`

Υποστήριξη Συναρτήσεων

- Κατανομή μνήμης για επιπρόσθετα δεδομένα στη Heap



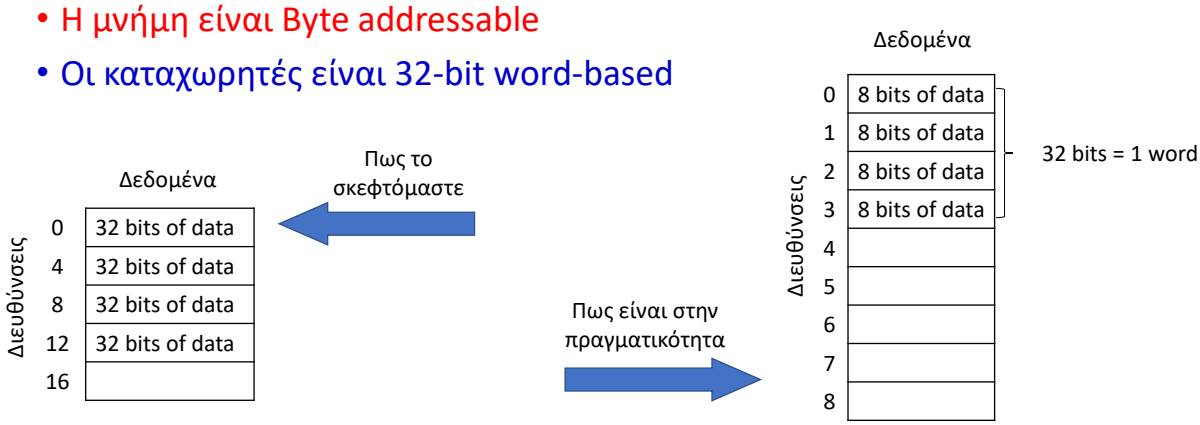
Heap: Κομμάτι της μνήμης που παραχωρείται για δυναμικά δεδομένα (dynamic data) (local variables, malloc())

Static: Κομμάτι της μνήμης που παραχωρείται για global δεδομένα του προγράμματος

Εντολές του προγράμματος που εκτελείται

Πως βλέπουμε τη Μνήμη

- Η μνήμη είναι Byte addressable
- Οι καταχωρητές είναι 32-bit word-based



Επεξεργασία χαρακτήρων και κειμένου

- Η αρχιτεκτονική MIPS παρέχει εντολές χρήσιμες για επεξεργασία κειμένου
- Θυμάστε τον κώδικα ASCII; 8-bit για αναπαράσταση χαρακτήρων
  - `lb` (load byte): φορτώνει ένα byte από τη μνήμη στα 8 πιο δεξιά bits ενός καταχωρητή
  - `sb` (store byte): αποθηκεύει τα 8 πιο δεξιά bits ενός καταχωρητή στη μνήμη
- Παράδειγμα:

```
lb $t0, 0($sp)        # διαβάζει 1 byte (= 1 χαρακτήρας) από $sp+0 στο $t0
sb $t0, 0($gp)        # γράφει 1 byte από $t0 (= 1 χαρακτήρας) στη $gp+0
```

- Η MIPS επίσης υποστηρίζει εντολές για φόρτωση μισών λέξεων (half words = 16 bits), χρήσιμες για τον κώδικα Unicode που χρησιμοποιεί η Java

```
lh $t0, 0($sp)        # διαβάζει 2 bytes (= 1 χαρακτήρας) από $sp+0 στο $t0
sh $t0, 0($gp)        # γράφει 2 bytes από $t0 (= 1 χαρακτήρας) στη $gp+0
```

Επεξεργασία χαρακτήρων και κειμένου

ASCII value	Char-acter	ASCII value	Char-acter	ASCII value	Char-acter	ASCII value	Char-acter	ASCII value	Char-acter	ASCII value	Char-acter
32	space	48	0	64	@	80	P	96	`	112	p
33	!	49	1	65	A	81	Q	97	a	113	q
34	"	50	2	66	B	82	R	98	b	114	r
35	#	51	3	67	C	83	S	99	c	115	s
36	$	52	4	68	D	84	T	100	d	116	t
37	%	53	5	69	E	85	U	101	e	117	u
38	&	54	6	70	F	86	V	102	f	118	v
39	'	55	7	71	G	87	W	103	g	119	w
40	(	56	8	72	H	88	X	104	h	120	x
41	)	57	9	73	I	89	Y	105	i	121	y
42	*	58	:	74	J	90	Z	106	j	122	z
43	+	59	;	75	K	91	[	107	k	123	{
44	,	60	<	76	L	92	\	108	l	124	|
45	-	61	=	77	M	93	]	109	m	125	}
46	.	62	>	78	N	94	^	110	n	126	~
47	/	63	?	79	O	95	_	111	o	127	DEL

FIGURE 2.21 ASCII representation of characters. Note that upper- and lowercase letters differ by exactly 32; this observation can lead to short-cuts in checking or changing upper- and lowercase. Values not shown include formatting characters. For example, 8 represents backspace, 9 represents a tab character, and 13 a carriage return. Another useful value is 0 for null, the value the programming language C uses to mark the end of a string.

Παράδειγμα: String Copy

- C code:
  - Null-terminated string

```
void strcpy (char x[], char y[])
{ int i;
  i = 0;
  while ((x[i]=y[i])!='\0')
    i += 1;
}
```

  - Η διεύθυνση των x, y → $a0, $a1
  - i → $s0

Παράδειγμα: String Copy

- MIPS code:

```
strcpy:
    addi $sp, $sp, -4       # adjust stack for 1 item
    sw   $s0, 0($sp)        # save $s0
    add  $s0, $zero, $zero  # i = 0
L1: add  $t1, $s0, $a1      # addr of y[i] in $t1
    lb   $t2, 0($t1)        # $t2 = y[i]
    add  $t3, $s0, $a0      # addr of x[i] in $t3
    sb   $t2, 0($t3)        # x[i] = y[i]
    beq  $t2, $zero, L2     # exit loop if y[i] == 0
    addi $s0, $s0, 1        # i = i + 1
    j    L1                 # next iteration of loop
L2: lw   $s0, 0($sp)        # restore saved $s0
    addi $sp, $sp, 4        # pop 1 item from stack
    jr   $ra                # and return
```


Παράδειγμα Assembly #1

```
int sum_array(int arr[], int size) {
    int sum = 0;

    for (int i = 0; i < size; i++) {
        sum += arr[i];
    }

    return sum;
}
```

sum -> $s0

Παράδειγμα Assembly #1

```
sum_array:
    # Save $s0 to stack (callee-saved register)
    addi $sp, $sp, -4        # Allocate stack space
    sw   $s0, 0($sp)         # Save old value of $s0

    # Initialize sum = 0 and i = 0
    add  $t0, $zero, $zero    # i = 0
    add  $s0, $zero, $zero    # sum = 0

loop:
    slt  $t3, $t0, $a1       # Set $t3 = 1 if (i < size)
    beq  $t3, $zero, end     # If (i >= size), exit loop

    # Load arr[i] into $t2
    sll  $t3, $t0, 2         # Multiply i by 4 (word size)
    add  $t3, $a0, $t3       # Address of arr[i]
    lw   $t2, 0($t3)         # Load arr[i] into $t2

    # sum += arr[i]
    add  $s0, $s0, $t2

    # i++
    addi $t0, $t0, 1
    j    loop                # Repeat loop

end:
    add $v0, $s0, $zero      # Move sum to return register

    # Restore $s0 from stack
    lw   $s0, 0($sp)         # Restore old value of $s0
    addi $sp, $sp, 4         # Free stack space

    jr   $ra                 # Return to caller
```


Παράδειγμα Assembly #2

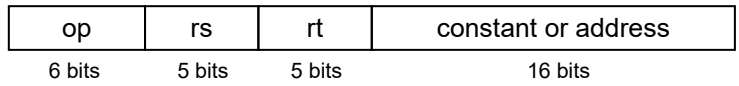
```
int sum = 0;

for (int i = 0; i < size; i++) {
    sum += arr[i];
}
```

Παράδειγμα Assembly #2

```
# Initialize sum = 0 and i = 0
add  $t0, $zero, $zero    # i = 0
add  $s0, $zero, $zero    # sum = 0
loop:
    slt  $t3, $t0, $a1       # Set $t3 = 1 if (i < size)
    beq  $t3, $zero, end     # If (i >= size), exit loop
    # Load arr[i] into $t2
    sll  $t3, $t0, 2         # Multiply i by 4 (word size)
    add  $t3, $a0, $t3       # Address of arr[i]
    lw   $t2, 0($t3)         # Load arr[i] into $t2
    # sum += arr[i]
    add  $s0, $s0, $t2
    # i++
    addi $t0, $t0, 1
    j    loop                # Repeat loop
end:…
```

Σταθερές και διευθύνσεις 32-bit



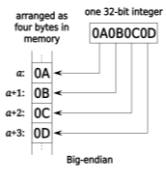
- Οι περισσότερες σταθερές είναι μικρές, άρα 16-bit είναι αρκετά
- Πώς φορτώνουμε (load) σταθερές > 16-bits?
- Χρειαζόμαστε 2 εντολές:
  - Load Upper Immediate (φορτώνει στα 16 πιο σημαντικά bits)

```
lui $t0, 1010101010101010
```

- Για να πάρουμε τα 16 λιγότερο σημαντικά bits χρησιμοποιούμε την λογική εντολή (or immediate):

```
ori $t0, $t0, 0000000011111111
```


Παράδειγμα




```
0000 0000 0011 1101 | 0000 1001 0000 0000
        61                  2304
```

Στη MIPS, ο assembler είναι υπεύθυνος για τη δημιουργία των «μεγάλων» σταθερών ή διευθύνσεων (long constants/addresses) χρησιμοποιώντας τον ειδικό καταχωρητή **$at**

```
lui $s0, 61   # 61 decimal = 0000 0000 0011 1101 binary

0000 0000 0011 1101 0000 0000 0000 0000

ori $s0, $s0, 2304 # 2304 decimal = 0000 1001 0000 0000

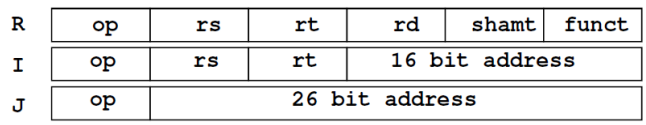
0000 0000 0011 1101 0000 1001 0000 0000
```

Κώδικας Assembly vs. Γλώσσα Μηχανής

- Η Assembly παρέχει συμβολική αναπαράσταση των εντολών, για λόγους διευκόλυνσης του προγραμματιστή
  - Πολύ πιο εύκολο από το να γράφουμε με δυαδικούς / δεκαεξαδικούς αριθμούς
  - Τυποποίηση, π.χ. ο καταχωρητής του αποτελέσματος πάντα πρώτος
- Η γλώσσα μηχανής (Machine language) είναι η πραγματικότητα που κρύβεται πίσω από την Assembly
  - Π.χ. ο καταχωρητής του αποτελέσματος ΔΕΝ είναι πάντα πρώτος
- Η Assembly παρέχει «ψευδοεντολές» (pseudoinstructions)
  - Π.χ., η εντολή `move $t0, $t1` υπάρχει μόνο στην Assembly
  - Η υλοποίηση της (== γλώσσα μηχανής) γίνεται με την εντολή
    `add $t0, $t1, $zero`
- Για ανάλυση επίδοσης πρέπει να μετρούμε τον αριθμό των πραγματικών εντολών, όχι των ψευδοεντολών

Επισκόπηση MIPS για διατάξεις εντολών

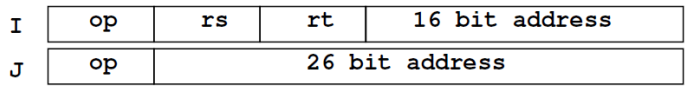
- Απλές εντολές, όλες των 32 bits
- Δομημένες εντολές, τυποποίηση
- Μόνο τρεις διαφορετικές διατάξεις εντολών



- Βασιζόμαστε στον μεταγλωττιστή για επίτευξη επίδοσης
- Πρέπει να βοηθούμε τον μεταγλωττιστή, όπου χρειάζεται, μέσω σωστού προγραμματισμού

Διευθύνσεις για Branches και Jumps

- Εντολές:
  - `bne $t4, $t5, Label  # Επόμενη εντολή στο Label εάν $t4≠$t5`
  - `beq $t4, $t5, Label  # Επόμενη εντολή στο Label εάν $t4=$t5`
  - `j Label  # Επόμενη εντολή στο Label`
- Διατάξεις εντολών:



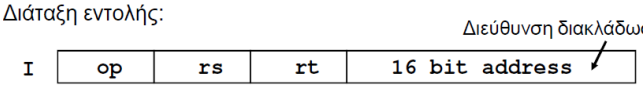
- Οι διευθύνσεις στις δύο πιο πάνω διατάξεις δεν μπορούν να είναι 32 bits (μέγιστο 2^{16} για I-type και 2^{26} για J-type)
  - Υποδηλώνει ότι κανένα πρόγραμμα δεν μπορεί να είναι μεγαλύτερο από 2^{16} bytes ή 2^{26} bytes
  - Η μνήμη στη MIPS έχει 2^{32} bytes (ή 2^{30} words = 4 GB)
- Πως μπορούμε να έχουμε πρόσβαση σε ολόκληρη τη μνήμη από εντολές μεταφοράς δεδομένων (load/store);

Διευθύνσεις για Branches

- Εντολές διακλάδωσης υπό συνθήκη :
  - `bne $t4, $t5, Label  # Επόμενη εντολή στο Label εάν $t4≠$t5`
  - `beq $t4, $t5, Label  # Επόμενη εντολή στο Label εάν $t4=$t5`

το Label είναι ουσιαστικά διεύθυνση που θα αποφασιστεί σε μεταγενέστερο στάδιο από τον μεταγλωττιστή

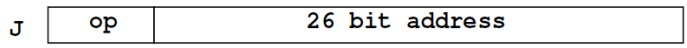
Διάταξη εντολής:



- Μπορούμε να καθορίσουμε ένα καταχωρητή του οποίου τα δεδομένα θα προστεθούν στην διεύθυνση της διακλάδωσης (όπως γίνεται για τις εντολές lw και sw). Έτσι, η εντολή διακλάδωσης θα υπολογίσει:
  - PC = τιμή καταχωρητή + διεύθυνση διακλάδωσης
  - Αυτό επιτρέπει σε ένα πρόγραμμα να είναι μέχρι και 2^{32} bytes και συγχρόνως να μπορεί να χρησιμοποιεί διακλαδώσεις υπό συνθήκη
  - Αυτός ο τρόπος διευθυνσιοδότησης ονομάζεται **relative addressing mode** (η πραγματική διεύθυνση διακλάδωσης υπολογίζεται σχετικά με τα δεδομένα ενός καταχωρητή)
  - Οι περισσότερες διακλαδώσεις υπό συνθήκη είναι τοπικές και επομένως χρησιμοποιούμε:
  - PC-relative addressing: PC = PC + διεύθυνση διακλάδωσης

Διευθύνσεις για Jumps

- Εντολή άλματος (jump):
  - `j Label` `# Επόμενη εντολή στο Label`
  - `jal Label` `# Καλεί την συνάρτηση Label`



- Στη MIPS κάθε εντολή είναι 4 bytes και η μνήμη είναι byte-addressable. Στα jump instructions τα 26 bits του target μετατοπίζονται κατά 2 bits (λόγω word alignment), άρα μπορούν να προσδιορίσουν 2^{28} byte διευθύνσεις
  - Τα 2 τελευταία bits μιας διεύθυνσης εντολών είναι πάντα 00 (αφού προχωρούμε ανά 4 bytesστη μνήμη)
- Ψευδοάμεση Διευθυνσιοδότηση (Pseudodirect addressing)

  PC = 4 MSBs (Most Significant Bits) του PC [Concatenate] διεύθυνση άλματος
- Οι εντολές άλματος χρησιμοποιούν τα 4 πιο σημαντικά ψηφία (MSBs) του PC

Διευθύνσεις για Branches και Jumps

- Η MIPS αυξάνει την απόσταση των διακλαδώσεων και των αλμάτων περαιτέρω:
  - Η διεύθυνση MIPS είναι σχετική με την επόμενη εντολή (PC+4) αντί την τρέχουσα εντολή (PC)
  - Η διεύθυνση αναφέρεται σε λέξεις (words) αντί σε bytes
  - Έτσι, η διεύθυνση διακλάδωσης ή άλματος μπορεί να είναι 4 φορές μεγαλύτερη (2 more bits)
- PC-relative addressing γίνεται:
  - PC = (PC+4) + (διεύθυνση διακλάδωσης * 4)
- Pseudodirect addressing γίνεται:
  - PC = 4 MSBs του PC && (διεύθυνση άλματος * 4) , && → Concatenate

Παράδειγμα

- Θυμηθείτε την άσκηση: **`while (save[i] == k) i++;`**

```
Loop:   sll  $t1, $s3, 2        # $t1 = 4*i
        add  $t1, $t1, $s6      # $t1 = address of save[i]
        lw   $t0, 0($t1)        # $t0 = save[i]
        bne  $t0, $s5, Exit     # go to Exit if save[i] ≠ k
        addi $s3, $s3, 1        # i++
        j    Loop               # Jump back to the start of the loop
Exit: …
```

Μεταγλωττισμένες εντολές και οι διευθύνσεις αποθήκευσής τους

80000	0	0	19	9	2	0	`Loop: sll`
80004	0	9	22	9	0	32	`add`
80008	35	9	8	0			`lw`
80012	5	8	21	2			`bne`
80016	8	19	19	1			`addi`
80020	2	20000					`j`
80024	…						`Exit: …`

Παράδειγμα (συν.)

- PC-relative addressing για **`bne $t0, $s5, Exit`**
  - PC = (PC+4) + (branch address* 4)
  - PC = (80012+4) + (2*4) = 80024
- Pseudodirect addressing για **`j Loop`**
  - PC = (4 MSBs of PC) + (διεύθυνση άλματος * 4)
  - PC = $(0000)_2$ && $(20000*4)_{10} = 80000_{10}$ && → Concatenate

Μεταγλωττισμένες εντολές και οι διευθύνσεις αποθήκευσής τους

80000	0	0	19	9	2	0	`Loop: sll`
80004	0	9	22	9	0	32	`add`
80008	35	9	8	0			`lw`
80012	5	8	21	2			`bne`
80016	8	19	19	1			`addi`
80020	2	20000					`j`
80024	…						`Exit: …`

Εναλλακτικός τρόπος για μακρινή διακλάδωση

- Θεωρήστε ότι **`$s0 == $s1`**
- Αντικαταστήστε την πιο κάτω εντολή με μία ή περισσότερες εντολές έτσι ώστε να αυξηθεί η απόσταση της διακλάδωσης (branching distance)

```
beq $s0, $s1, L1
```



```
bne $s0, $s1, L2
j L1
L2:
```

ΓΙΑΤΙ??

Σύνοψη

MIPS operands

Name	Example	Comments
32 registers	`$s0–$s7`, `$t0–$t9`, `$zero`, `$a0–$a3`, `$v0–$v1`, `$gp`, `$fp`, `$sp`, `$ra`, `$at`	Fast locations for data. In MIPS, data must be in registers to perform arithmetic. MIPS register `$zero` always equals 0. Register `$at` is reserved for the assembler to handle large constants.
2^{30} memory words	Memory[0], Memory[4], . . . , Memory[4294967292]	Accessed only by data transfer instructions. MIPS uses byte addresses, so sequential word addresses differ by 4. Memory holds data structures, arrays, and spilled registers, such as those saved on procedure calls.

Σύνοψη

MIPS assembly language

Category	Instruction	Example	Meaning	Comments
Arithmetic	add	`add $s1,$s2,$s3`	$s1 = $s2 + $s3	Three register operands
	subtract	`sub $s1,$s2,$s3`	$s1 = $s2 – $s3	Three register operands
	add immediate	`addi $s1,$s2,100`	$s1 = $s2 + 100	Used to add constants
Data transfer	load word	`lw $s1,100($s2)`	$s1 = Memory[$s2 + 100]	Word from memory to register
	store word	`sw $s1,100($s2)`	Memory[$s2 + 100] = $s1	Word from register to memory
	load half	`lh $s1,100($s2)`	$s1 = Memory[$s2 + 100]	Halfword memory to register
	store half	`sh $s1,100($s2)`	Memory[$s2 + 100] = $s1	Halfword register to memory
	load byte	`lb $s1,100($s2)`	$s1 = Memory[$s2 + 100]	Byte from memory to register
	store byte	`sb $s1,100($s2)`	Memory[$s2 + 100] = $s1	Byte from register to memory
	load upper immed.	`lui $s1,100`	$s1 = 100 * 2^{16}	Loads constant in upper 16 bits
Logical	and	`and $s1,$s2,$s3`	$s1 = $s2 & $s3	Three reg. operands; bit-by-bit AND
	or	`or $s1,$s2,$s3`	$s1 = $s2 | $s3	Three reg. operands; bit-by-bit OR
	nor	`nor $s1,$s2,$s3`	$s1 = ~ ($s2 | $s3)	Three reg. operands; bit-by-bit NOR
	and immediate	`andi $s1,$s2,100`	$s1 = $s2 & 100	Bit-by-bit AND reg with constant
	or immediate	`ori $s1,$s2,100`	$s1 = $s2 | 100	Bit-by-bit OR reg with constant
	shift left logical	`sll $s1,$s2,10`	$s1 = $s2 << 10	Shift left by constant
	shift right logical	`srl $s1,$s2,10`	$s1 = $s2 >> 10	Shift right by constant
Conditional branch	branch on equal	`beq $s1,$s2,25`	if ($s1 == $s2) go to PC + 4 + 100	Equal test; PC-relative branch
	branch on not equal	`bne $s1,$s2,25`	if ($s1 != $s2) go to PC + 4 + 100	Not equal test; PC-relative
	set on less than	`slt $s1,$s2,$s3`	if ($s2 < $s3) $s1 = 1; else $s1 = 0	Compare less than; for `beq`, `bne`
	set less than immediate	`slti $s1,$s2,100`	if ($s2 < 100) $s1 = 1; else $s1 = 0	Compare less than constant
Uncondi-tional jump	jump	`j 2500`	go to 10000	Jump to target address
	jump register	`jr $ra`	go to $ra	For switch, procedure return
	jump and link	`jal 2500`	$ra = PC + 4; go to 10000	For procedure call

Τρόποι Καθορισμού Διεύθυνσης

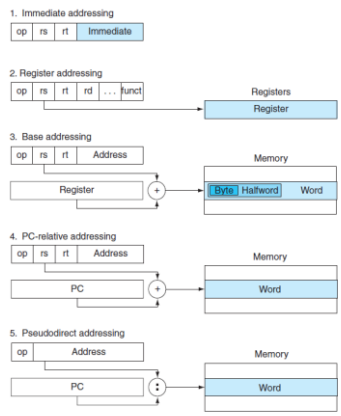
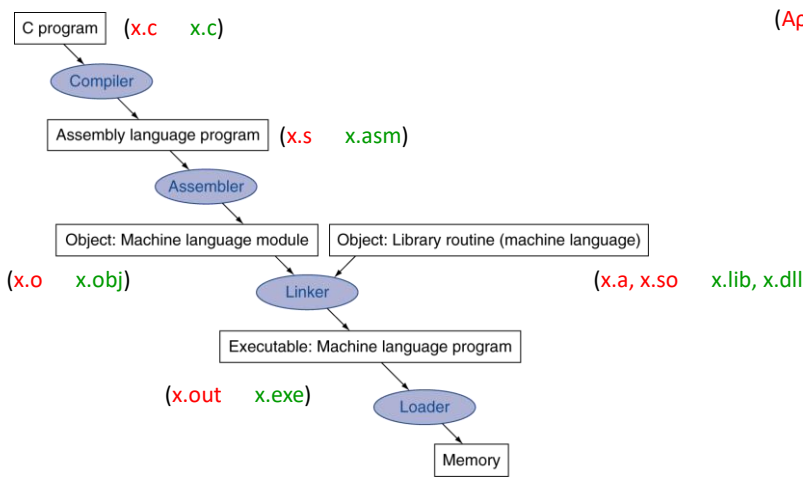
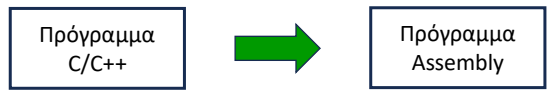


FIGURE 2.24 Illustration of the five MIPS addressing modes. The operands are shaded in color. The operand of mode 3 is in memory, whereas the operand for mode 2 is a register. Note that versions of load and store access bytes, halfwords, or words. For mode 1, the operand is 16 bits of the instruction itself. Modes 4 and 5 address instructions in memory, with mode 4 adding a 16-bit address shifted left 2 bits to the PC and mode 5 concatenating a 26-bit address shifted left 2 bits with the 4 upper bits of the PC.

Μετάφραση και Εκτέλεση Προγράμματος



Compiler (Μεταγλωττιστής)



- Μεταφράζει από γλώσσα υψηλού επιπέδου σε κώδικα assembly

Assembler (Συμβολομεταφραστής)

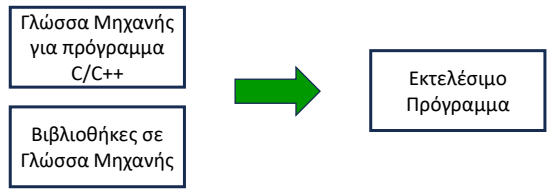


- Διασύνδεση μεταξύ του λογισμικού και του υλικού ενός υπολογιστή
- Μπορούμε να χρησιμοποιήσουμε ψευδο-εντολές, όπως:
  - `move $t0, $t1` → `add $t0, $zero, $t1`
  - `blt $t0, $t1, L` → `slt $at, $t0, $t1`
    `bne $at, $zero, L`
  - `mult` → `sll`
- Ο assembler της MIPS χρησιμοποιεί το δεκαεξαδικό σύστημα αριθμών

Assembler (Συμβολομεταφραστής)

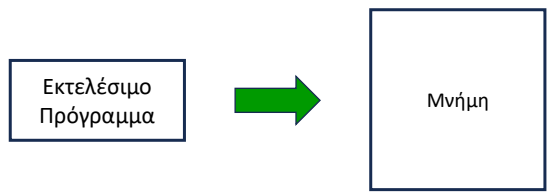
- Το αρχείο του εκτελέσιμου κώδικα περιέχει (στο Unix):
  1. Header: καθορίζει το μέγεθος/τοποθεσία των υπολοίπων μερών
  2. Text segment (Κείμενο): περιέχει τη γλώσσα μηχανής (εκτελέσιμο κώδικα)
  3. Static data segment (Στατικά δεδομένα): τα στατικά δεδομένα του προγράμματος
  4. Relocation information (Πληροφορίες μετατόπισης): προσδιορίζει λέξεις εντολών/δεδομένων βάση απόλυτων διευθύνσεων
  5. Symbol table (Πίνακας συμβόλων): ταιριάζει τα ονόματα των labels με απόλυτες διευθύνσεις (word addresses) στη μνήμη
  6. Debugging information: περιγραφή του τρόπου μεταγλωττισμού ώστε να μπορεί να συσχετιστεί η γλώσσα της μηχανής με τις εντολές γλώσσας υψηλού επιπέδου (π.χ. C)

Linker



- Συνδυάζει προγράμματα που έχουν μεταγλωττιστεί και συμβολομεταφραστεί ανεξάρτητα, σε ένα ενιαίο εκτελέσιμο κώδικα
- Επιτρέπει τη χρήση προκαθορισμένων και ήδη συμβολομεταφρασμένων βιβλιοθηκών κώδικα
- Επιλύει όλα τα ακαθόριστα labels (σε απόλυτες διευθύνσεις)
  - Χρησιμοποιεί τις πληροφορίες μετατόπισης και τον πίνακα συμβόλων που είναι μέρη του εκτελέσιμου αρχείου (object file)
- Καθορίζει την απόλυτη διεύθυνση στη μνήμη για την κάθε συνάρτηση / ενότητα

Loader



- Τοποθετεί το εκτελέσιμο πρόγραμμα (machine code με απόλυτες διευθύνσεις) στην κυρίως μνήμη
  - Πρέπει να εντοπίσει αρκετό χώρο στη μνήμη που να μπορεί να κρατήσει ολόκληρο το πρόγραμμα
  - Αντιγράφει τα δεδομένα από τον δίσκο στη μνήμη
  - Αντιγράφει τις παραμέτρους του προγράμματος στη στοίβα
  - Αρχικοποιεί τους καταχωρητές και θέτει τον δείκτη στοίβας
  - Πηδά στην διεύθυνση της αρχικής συνάρτησης

Static vs. Dynamic Libraries

Static Libraries

- Οι βιβλιοθήκες συνδέονται και φορτώνονται πριν την εκτέλεση του προγράμματος
- Γρηγορότερη εκτέλεση
- Καινούριες εκδόσεις πρέπει να ξανά-συνδεθούν και να ξανά-φορτωθούν
- Απαιτεί περισσότερο χώρο (φορτώνεται ολόκληρη η βιβλιοθήκη)

Dynamic Libraries

- Οι βιβλιοθήκες ΔΕΝ συνδέονται ΟΥΤΕ φορτώνονται ΜΕΧΡΙ τη στιγμή που εκτελείται το πρόγραμμα
- Πιο αργή εκτέλεση
- Οι καινούριες εκδόσεις υποστηρίζονται αυτόματα
- Μειωμένος χώρος

Lazy Linkage: μια ρουτίνα συνδέεται μόνο αφότου έχει καλεστεί, έτσι δεν φορτώνονται αχρησιμοποίητες ρουτίνες

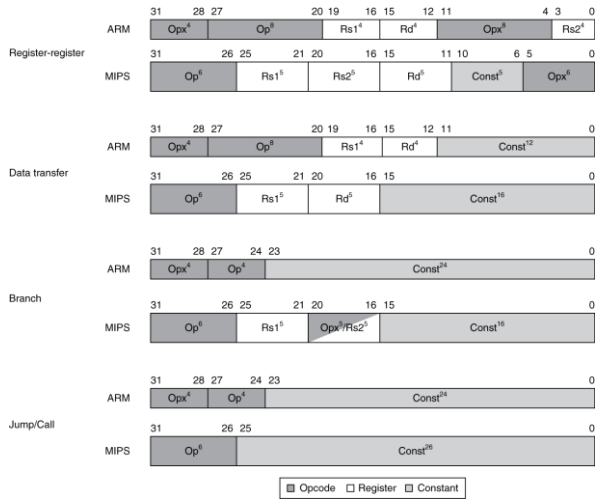
Άλλες αρχιτεκτονικές ISA

ARM ISA

- Η πιο κοινή για embedded επεξεργαστές (κινητές συσκευές, etc.)
- Η πρώτη έκδοση είχε ένα μικρό σύνολο από εντολές που ήταν αρκετά παρόμοια με την MIPS

	ARM	MIPS
Date announced	1985	1985
Instruction size	32 bits	32 bits
Address space	32-bit flat	32-bit flat
Data alignment	Aligned	Aligned
Data addressing modes	9	3
Registers	15 × 32-bit	31 × 32-bit
Input/output	Memory mapped	Memory mapped

Κωδικοποίηση εντολών



ARM v8 Instructions

- Τεράστια αλλαγή της ISA όταν πήγε σε 64-bit
- ARM v8 resembles MIPS
  - Changes from v7:
    - No conditional execution field
    - Immediate field is 12-bit constant
    - Dropped load/store multiple
    - PC is no longer a GPR
    - GPR set expanded to 32
    - Addressing modes work for all word sizes
    - Divide instruction
    - Branch if equal/branch if not equal instructions

Intel x86 ISA

- Evolution with backward compatibility
  - 8080 (1974): 8-bit microprocessor
    - Accumulator, plus 3 index-register pairs
  - 8086 (1978): 16-bit extension to 8080
    - Complex instruction set (CISC)
  - 8087 (1980): floating-point coprocessor
    - Adds FP instructions and register stack
  - 80286 (1982): 24-bit addresses, MMU
    - Segmented memory mapping and protection
  - 80386 (1985): 32-bit extension (now IA-32)
    - Additional addressing modes and operations
    - Paged memory mapping as well as segments

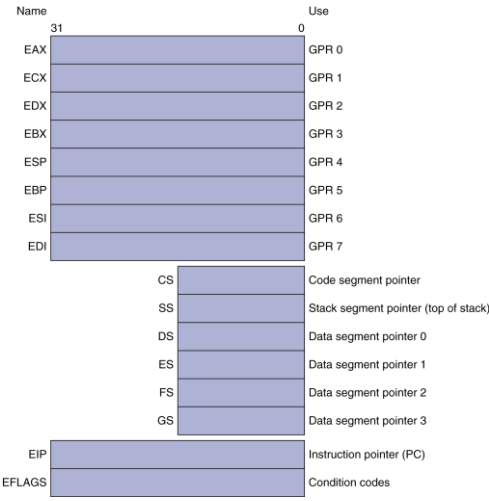
Intel x86 ISA

- Further evolution…
  - i486 (1989): pipelined, on-chip caches and FPU
    - Compatible competitors: AMD, Cyrix, …
  - Pentium (1993): superscalar, 64-bit datapath
    - Later versions added MMX (Multi-Media eXtension) instructions
    - The infamous FDIV bug
  - Pentium Pro (1995), Pentium II (1997)
    - New microarchitecture (see Colwell, *The Pentium Chronicles*)
  - Pentium III (1999)
    - Added SSE (Streaming SIMD Extensions) and associated registers
  - Pentium 4 (2001)
    - New microarchitecture
    - Added SSE2 instructions

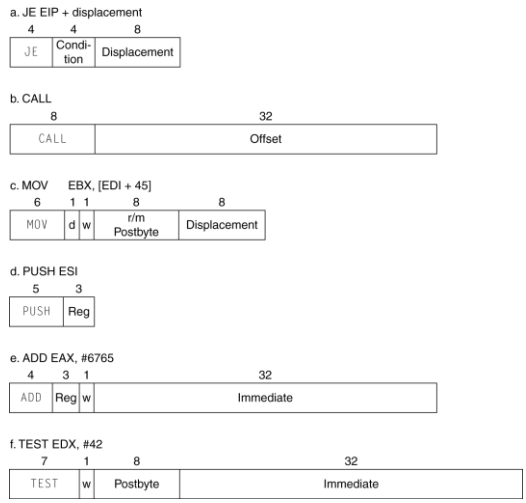
Intel x86 ISA

- And further…
  - AMD64 (2003): extended architecture to 64 bits
  - EM64T – Extended Memory 64 Technology (2004)
    - AMD64 adopted by Intel (with refinements)
    - Added SSE3 instructions
  - Intel Core (2006)
    - Added SSE4 instructions, virtual machine support
  - AMD64 (announced 2007): SSE5 instructions
    - Intel declined to follow, instead…
  - Advanced Vector Extension (announced 2008)
    - Longer SSE registers, more instructions
- If Intel didn't extend with compatibility, its competitors would!
  - Technical elegance ≠ market success

Basic x86 Registers



x86 Instruction Encoding

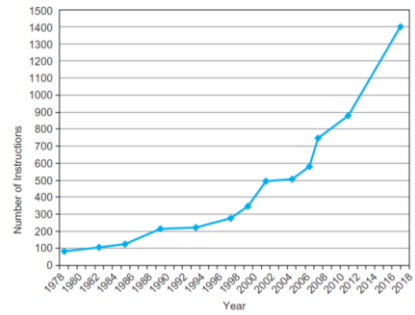


Fallacies

- Powerful instruction == higher performance?
  - Χρειάζονται πιο λίγες εντολές ναι αλλά
  - Πολύπλοκες εντολές είναι πιο δύσκολο να υλοποιηθούν και πιο αργές που μπορεί να καθυστερούν και τις υπόλοιπες
  - Οι compilers είναι πολύ καλοί σήμερα στο να φτιάχνουν γρήγορο κώδικα από πιο απλές εντολές!
- Χρησιμοποιούμε assembly για περισσότερη επίδοση?
  - Οι σημερινοί compilers είναι καλύτεροι στο να χειρίζονται καινούργιους επεξεργαστές!

Fallacies

- Backward compatibility (legacy code) == instruction set doesn't change?
  - Προσθέτουν όμως περισσότερες εντολές!
  - Μεγαλύτερο ISA



x86 instruction set

Ενότητα 02:
Εντολές:
Η Γλώσσα της Μηχανής

ΕΕΙ 233: ΟΡΓΑΝΩΣΗ ΥΠΟΛΟΓΙΣΤΩΝ

ΑΝΔΡΕΑΣ ΔΙΑΒΑΣΤΟΣ

andreas.diavastos@cut.ac.cy

Τεχνολογικό Πανεπιστήμιο Κύπρου
Τμήμα Ηλεκτρολόγων Μηχανικών και Μηχανικών Ηλεκτρονικών Υπολογιστών και Πληροφορικής

CARL

131